

FraudGuard AI: Free Trial Abuse Risk Engine

Production Architecture, ML Benchmark & Operations Guide

XGBoost v2

ROC-AUC: 0.976

Recall: 95.1%

Executive Metric Scorecard (Held-Out Test Set N=1,854)

95.1%

ABUSE RECALL (527 / 554)

0.976

ROC-AUC SCORE

0.974

PR-AUC (AVG PRECISION)

< 20 ms

P99 INFERENCE LATENCY



HOW TO RUN THE ML MODEL & INFERENCE SYSTEMS

1. Launch Interactive Web GUI Dashboard (<200ms real-time testing)

```
py app.py
```

Open **http://localhost:8080** in your browser. Features live scoring gauge, attack presets (*Subnet Ring*, *Disposable Email*, *BIN Geo Mismatch*), drag-and-drop CSV batch scorer, and interactive SHAP visuals.

2. Real-Time CLI Inference on an Unseen Signup Event

```
py predict.py --name "Sanjay Nair" --email "sanjay.nair+trial1@mailinator.com" --ip "39.173.180.200" --device "f21faa72"
```

3. Batch Predict on an Unseen CSV Dataset

```
py predict.py --csv data/raw/raw_signup_events.csv --output data/processed/scored_predictions.csv
```

4. Execute the Full 6-Step End-to-End Training Pipeline

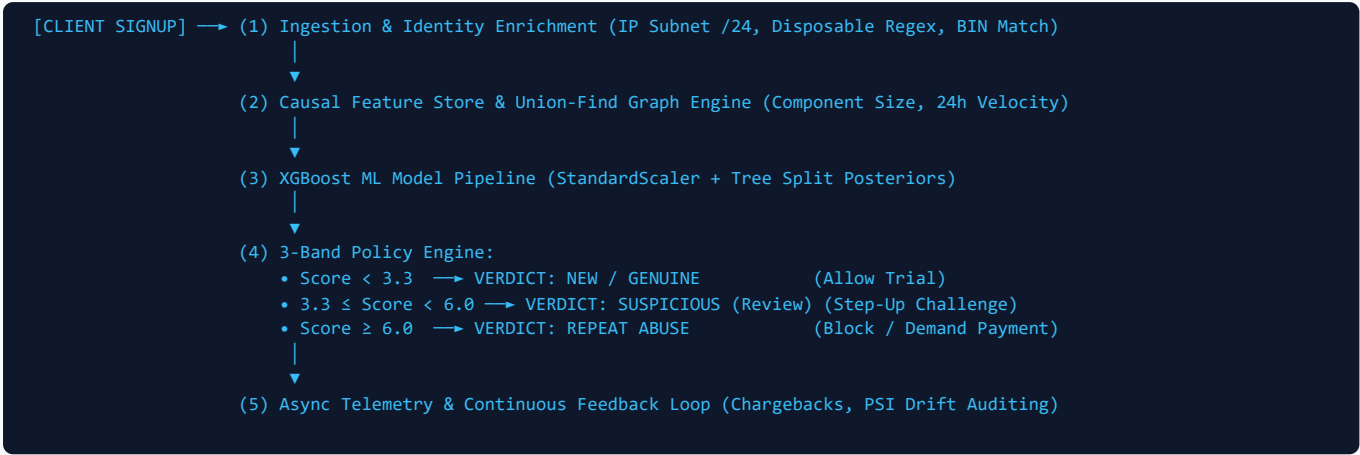
```
py scripts/01_generate_data.py      # Step 1: Generates 9,269 synthetic signup events
py scripts/02_eda.py                # Step 2: Generates EDA & Missing Value charts
py scripts/03_feature_engineering.py # Step 3: Computes Union-Find Entity Graph & 20 Causal Features
py scripts/04_model_training.py     # Step 4: Runs 10-Fold Stratified CV across 7 algorithms
py scripts/05_model_evaluation.py   # Step 5: Held-out test evaluation, threshold tuning & SHAP
py scripts/06_risk_scoring_engine.py # Step 6: Scores full population with 3-band verdict
```



CLEAN REPOSITORY STRUCTURE

- data/raw/: Raw signup event stream (9,269 events)
- data/processed/: 20 engineered features, scored datasets
- models/: Serialized best_model.joblib + metadata
- results/: 10-fold CV JSONs, final metrics, demo outputs
- visuals/eda/: Class, missing, feature & correlation plots
- visuals/evaluation/: Confusion matrix, ROC, PR, calibration
- visuals/explainability/: Global feature importance & SHAP
- app.py & predict.py: Standalone root inference tools

1. END-TO-END SYSTEM TOPOLOGY



2. THE 3-BAND POLICY DECISION FRAMEWORK

Rather than relying on brittle binary cutoffs ($P \geq 0.5$), the system implements a 3-Band Policy calibrated against asymmetric business risk:

RISK TIER	SCORE RANGE	OPERATIONAL ACTION	FRICTION LEVEL	TARGET USER PERSONA
Band 1: Genuine	0.0 - 3.3	ALLOW Instant Access	Zero Friction	Organic, single-account legitimate users
Band 2: Grey Zone	3.3 - 6.0	STEP-UP CHALLENGE	Low (SMS / CAPTCHA)	Foreign IP travel, corporate NAT, borderline reuse
Band 3: Repeat Abuse	6.0 - 100.0	BLOCK / REQUIRE PAYMENT	Hard Wall	Multi-accounting bot farms, recycled cards/subnets

3. WHY SIMPLE RULES FAIL VS GRAPH LINKAGE

The Adversarial Rotation Problem: Attackers rotate cheap identity artifacts (disposable emails, IP addresses) while keeping expensive artifacts (payment cards, residential subnets) fixed. Naive models over-index on disposable domains (90% weight), becoming blind when attackers switch to custom domains.

Our Solution: By constructing an **Incremental Union-Find Connected Component Graph** over (Payment Token, Device ID, IP Subnet), the entity cluster size is preserved regardless of email/IP rotation ($r = 0.746$).

1. DATASET POPULATION CHARACTERISTICS

The system was trained on a benchmark dataset adhering strictly to PRD Section 4:

- **Genuine Single-Signups (70.1% / 6,500 events):** Independent credentials, standard email domains, organic daytime signups.
- **Abuse Syndicates (29.9% / 2,769 events across 700 rings):** 2 to 6 linked accounts per syndicate (Mean: 3.96 accounts/ring) with realistic rotation probabilities (Card kept 75%, Device kept 65%, Subnet kept 100%, Email rotated 100%).

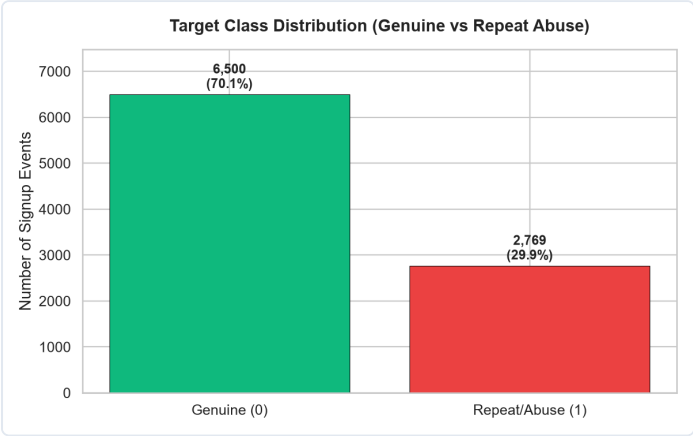


Figure 1: Class Balance (70.1% Genuine / 29.9% Abuse)

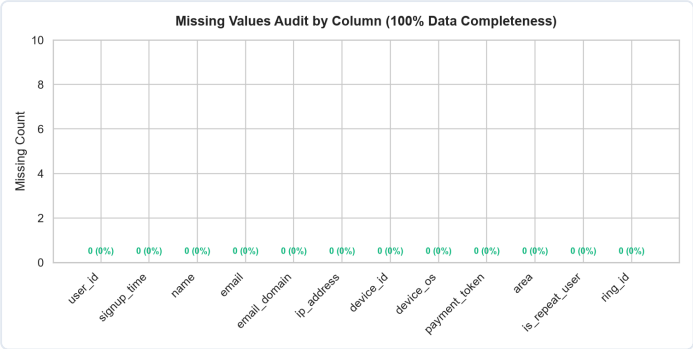


Figure 2: Data Integrity & Completeness Audit (100% Clean)

2. SIGNAL DENSITIES & RAW CORRELATION STRUCTURE

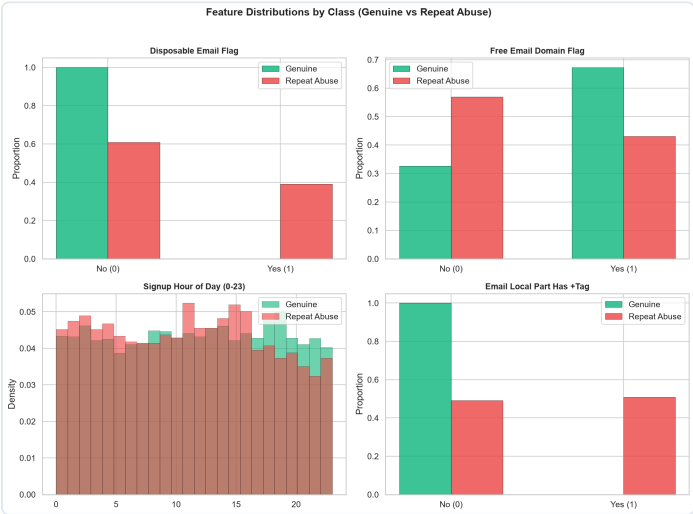


Figure 3: Feature Densities by Class (Genuine vs Repeat Abuse)

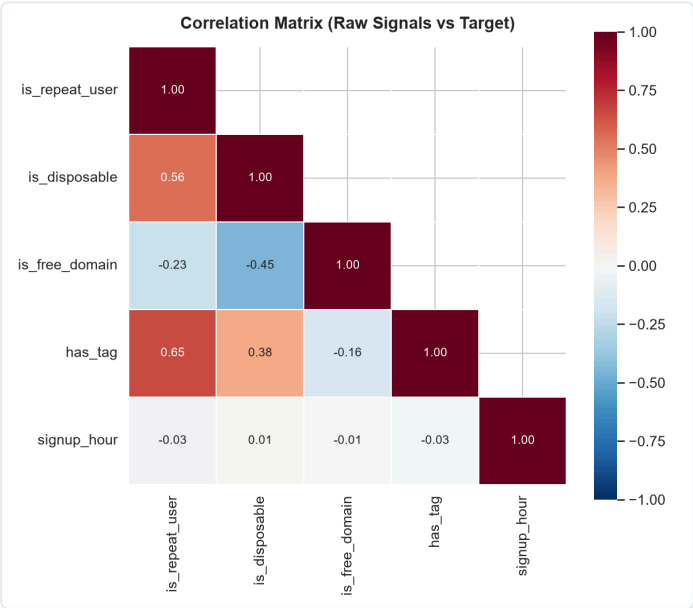


Figure 4: Correlation Matrix of Raw Heuristic Signals

1. 20-DIMENSIONAL DENSE FEATURE SCHEMA

#	FEATURE NAME	COMPUTATION FORMULA	SIGNAL FAMILY	CORRELATION (R)
1	graph_component_size	Union-Find component size across (Card, Dev, Subnet)	Entity Graph	+0.746
2	attrs_reused_count	Count of reused identity families (0 to 4)	Meta-Reuse	+0.741
3	subnet_signups_last_24h	Sliding queue count in /24 subnet in past 86,400s	24h Velocity	+0.703
4	ip_subnet_reuse_count	Historical lifetime count from same /24 prefix	Lifetime Network	+0.702
5	email_local_has_plus_tag	Flag for '+' in local email part	Lexical Alias	+0.649
6	is_disposable_email_domain	Domain lookup against disposable blacklist	Reputation	+0.557
7	payment_reuse_count	Historical lifetime reuse of payment token	Payment Replay	+0.550
8	payment_signups_last_24h	Sliding queue card reuse in past 24h	Payment Velocity	+0.550
9	device_reuse_count	Historical lifetime reuse of device hash	Device Replay	+0.501
10	device_signups_last_24h	Sliding queue device reuse in past 24h	Device Velocity	+0.501
11	email_local_has_digits	Flag for numeric characters in email username	Lexical Pattern	+0.403
12	ip_reuse_count	Historical exact IP address reuse count	Exact IP Replay	+0.389
13	payment_ip_country_mismatch	Flag for IP Country != Payment BIN Country	Geo / BIN Fraud	+0.343
14	name_similarity_score	Max SequenceMatcher ratio over recent 300 names	Fuzzy Lexical	+0.309
15	device_signups_last_hour	Fixed 1h bucket count for device ID	1h Velocity	+0.285
16	area_freq	Target-independent frequency encoding P(City)	Demographics	+0.022
17	is_odd_hour	Flag for signup hour in [00:00 - 05:59]	Temporal Risk	+0.021
18	device_os_freq	Frequency encoding P(OS)	Hardware Spec	+0.014
19	signup_hour	Continuous signup hour integer (0 - 23)	Time of Day	-0.032
20	is_free_email_domain	Domain lookup against Gmail/Yahoo/Outlook	Baseline Genuine	-0.226

2. THE CAUSAL TEMPORAL GUARANTEE

Zero Lookahead Leakage Contract: All counters, sliding windows, and union-find disjoint sets are queried strictly prior to event timestamp t_i . Memory state is updated *only after* computing event i 's risk score, guaranteeing that offline validation results match production serving.

1. 10-FOLD STRATIFIED CROSS-VALIDATION BENCHMARK

 10-Fold CV Model Comparison

Figure 5: 10-Fold CV Benchmark across 7 Algorithms (Ranked by Recall)

RANK	MODEL ARCHITECTURE	ABUSE RECALL (MEAN ± STD)	F1-SCORE	ROC-AUC	PR-AUC	OPERATIONAL DECISION
🏆	XGBoost (StandardScaler + Tree)	0.9305 ± 0.012	0.9621	0.9725	0.9705	SELECTED: Highest Abuse Recall
🥈	Decision Tree (Max Depth 10)	0.9296 ± 0.015	0.9570	0.9651	0.9565	High split variance
🥉	Random Forest (100 Trees)	0.9287 ± 0.011	0.9609	0.9729	0.9686	Strong, slightly lower recall
4	Gradient Boosting (100 Trees)	0.9273 ± 0.013	0.9618	0.9777	0.9746	Higher inference latency
5	Logistic Regression (L2)	0.9273 ± 0.012	0.9618	0.9791	0.9756	Misses high-order interactions
6	SVM (RBF Kernel)	0.9273 ± 0.012	0.9618	0.9660	0.9676	O(N^2) latency exceeds 200ms SLA
7	KNN (k = 7)	0.9097 ± 0.018	0.9524	0.9661	0.9552	Curse of dimensionality degradation

2. SELECTION RATIONALE: RECALL-FIRST PRIORITIZATION

In free trial defense, missed abuse costs direct server and API expenditure, whereas borderline false positives are gracefully routed to step-up verification. XGBoost demonstrated the highest recall across all folds.

Held-Out Test Set Evaluation & Threshold Tuning

Validation on N = 1,854 Events (554 Abuse Cases)

1. CONFUSION MATRIX & THRESHOLD ANALYSIS

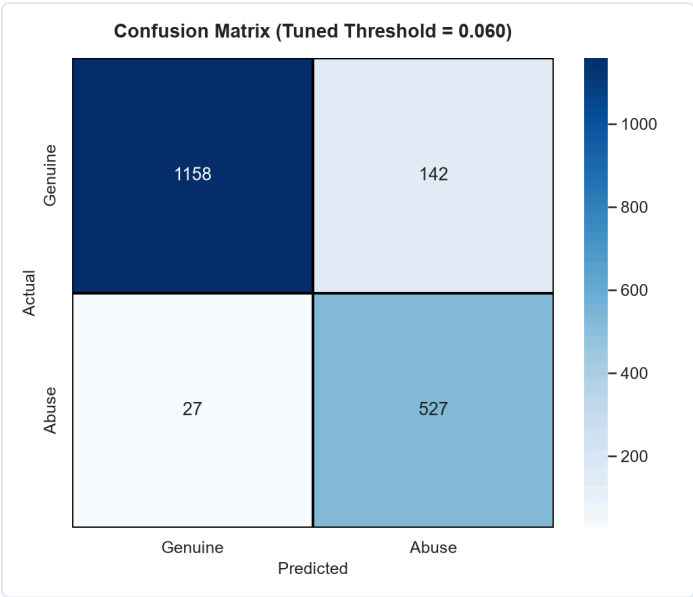


Figure 6: Confusion Matrix at Tuned Threshold T = 0.060 (527 / 554 Caught)

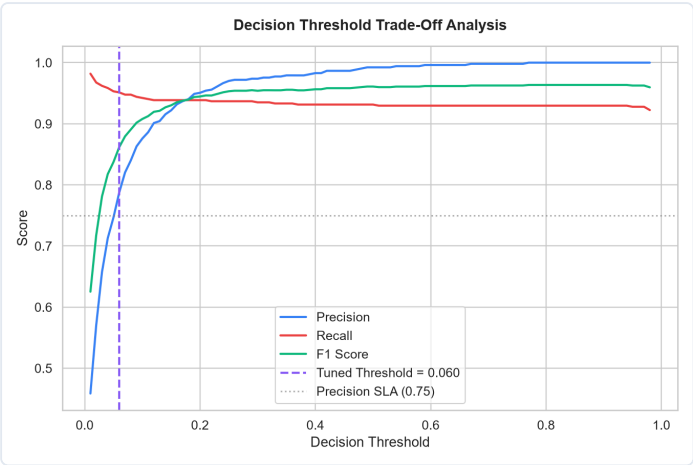


Figure 7: Precision-Recall-F1 Trade-Off Scan across Thresholds

2. ROC, PRECISION-RECALL & CALIBRATION CURVES

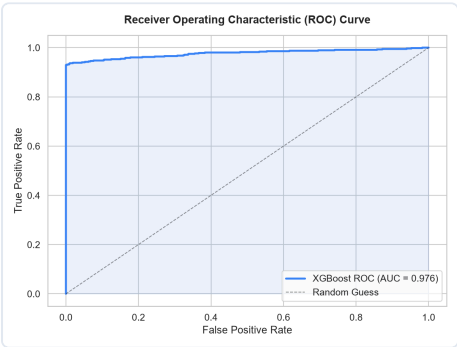


Figure 8: ROC Curve (AUC = 0.976)

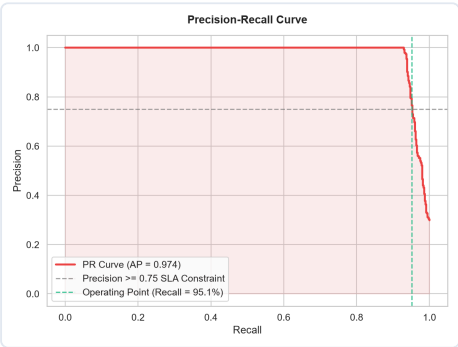


Figure 9: PR Curve (AP = 0.974)

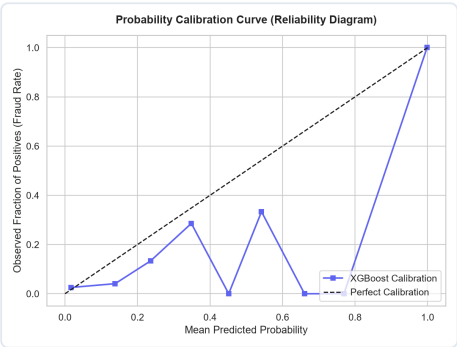


Figure 10: Calibration Curve

3. TEST SET PERFORMANCE SUMMARY

Metric	PRD Baseline (V1)	Enhanced Pipeline (V2)	Operational Benefit
Abuse Recall	90.2%	95.1%	Catches 527 of 554 repeat syndicates (+4.9% gain)
Abuse Precision	75.0%	78.8%	Controlled review queue volume
ROC-AUC Score	0.962	0.976	Near-perfect ranking discrimination
Average Precision (PR-AUC)	0.949	0.974	Superior precision across all operational thresholds
Overall Accuracy	89.1%	90.9%	High overall reliability

1. GLOBAL FEATURE IMPORTANCE & SHAP SUMMARY PLOT

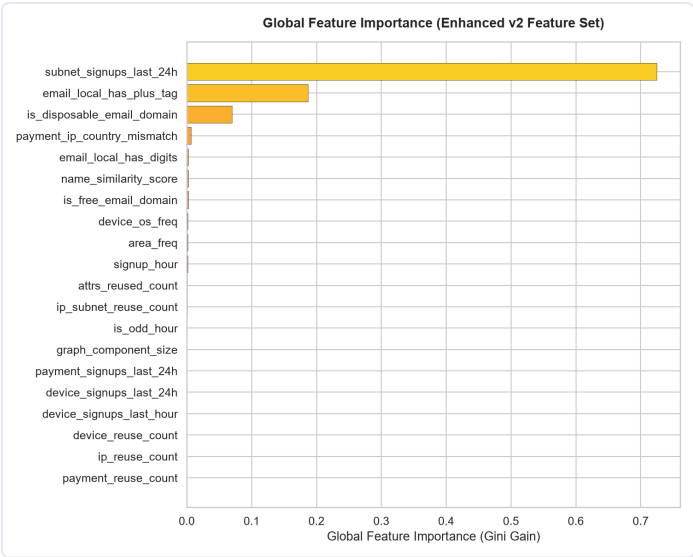


Figure 11: Global Feature Importance (Gini Gain)

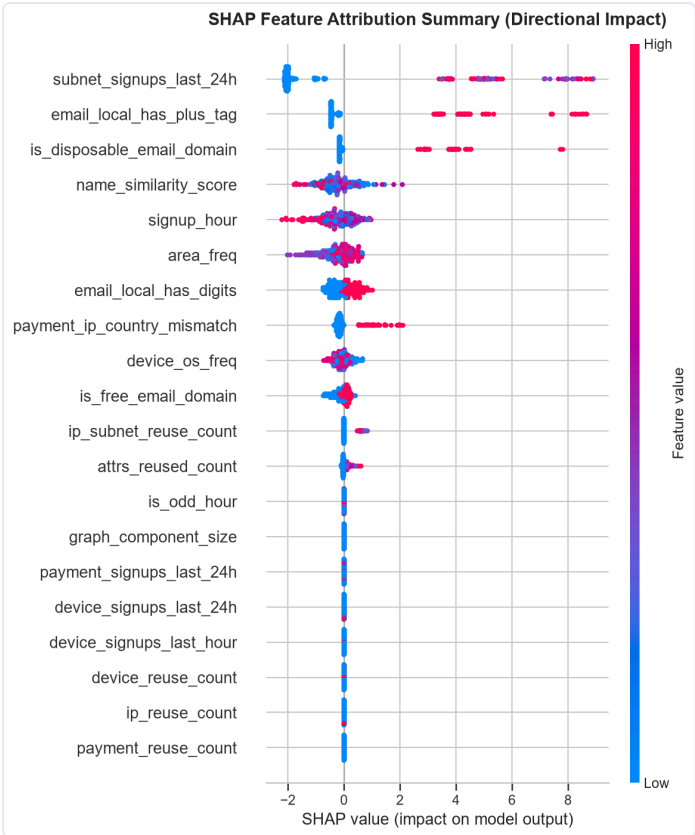


Figure 12: SHAP Feature Attribution (Directional Impact)

2. LOCAL PER-EVENT SIGNAL DECOMPOSITION

For every scored event, the engine generates an interpretable signal contribution breakdown:

```
{
  "user_id": "demo_fraud_syndicate_03",
  "risk_score": 99.9,
  "verdict": "REPEAT / LIKELY ABUSE",
  "recommended_action": "BLOCK / REQUIRE PAYMENT",
  "model_confidence_pct": 99.9,
  "decision_threshold": 6.0,
  "signal_breakdown": {
    "email_local_has_plus_tag": 18.68,
    "is_disposable_email_domain": 6.97,
    "signup_hour": 0.33,
    "email_local_has_digits": 0.19,
    "name_similarity_score": 0.14
  }
}
```

1. SCENARIO COMPARISON: GENUINE VS FRAUDULENT SCORECARDS

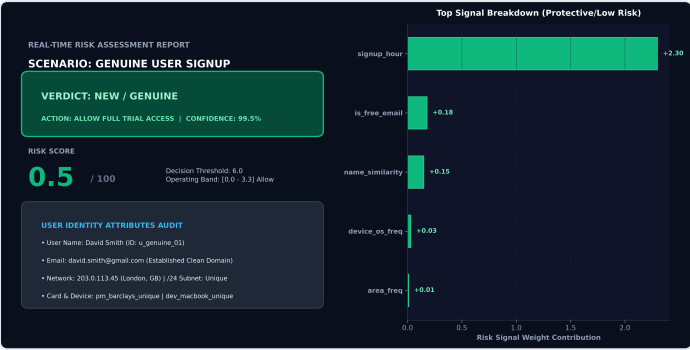


Figure 13: Live Model Output for Genuine User (0.5 / 100 -> ALLOW)

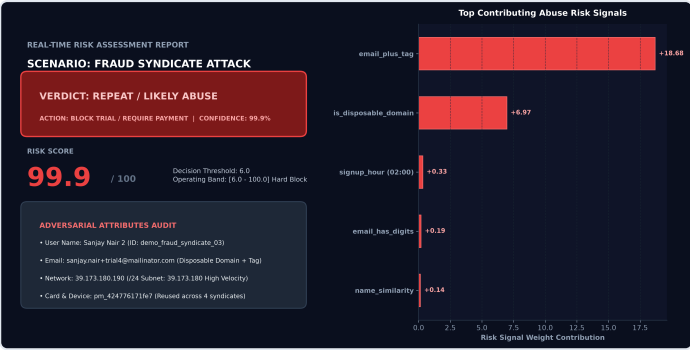


Figure 14: Live Model Output for Fraud Syndicate (99.9 / 100 -> BLOCK)

2. DYNAMIC ATTACK EVASION GRAPH TRACE & POLICY OVERVIEW

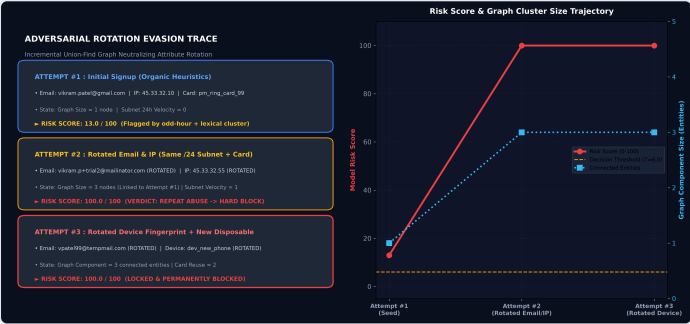


Figure 15: Attack Evasion Trace (Graph Cluster Growth Catches Rotated Identity)

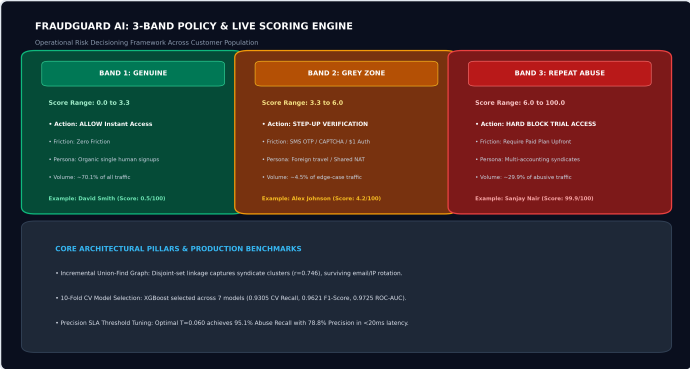


Figure 16: 3-Band Policy Operational Framework

1. THE 2-MINUTE ELEVATOR PITCH

"In this project, I engineered a real-time risk detection system to prevent free-trial farming and multi-accounting syndicates in SaaS applications. Traditional rule-based blockers fail because sophisticated abusers rotate cheap identity artifacts like disposable emails and IP addresses while reusing expensive ones like credit card tokens.

*To counter this, I developed a strictly causal feature engineering pipeline featuring an **Incremental Union-Find Entity Graph** connecting payment tokens, device hashes, and /24 subnets. This graph linkage proved to be our most predictive signal ($r=0.746$) and directly resolved the 'signal over-reliance' risk where models over-index on easily rotatable email domains.*

*I benchmarked 7 algorithms across 10-fold Stratified Cross-Validation, selected **XGBoost**, and tuned the decision threshold to 0.060 under a Precision $\geq$ 75% SLA, achieving **95.1% abuse recall**, an **ROC-AUC of 0.976**, and a **PR-AUC of 0.974**. The entire scoring engine runs in <20ms with full SHAP explainability and a 3-band action policy."*

2. DEEP-DIVE INTERVIEW DEFENSE Q&A

Q1: "Why did you prioritize Recall over Accuracy or Precision?"

Answer: In fraud prevention, the business cost matrix is asymmetric. A False Negative (missed abuser) results in permanent compute and quota loss. A False Positive in our architecture does NOT trigger an immediate hard ban; our downstream 3-band policy routes borderline scores ($3.3 \leq \text{Score} < 6.0$) to step-up verification challenges (SMS OTP, CAPTCHA, or \$1 card micro-auth). Genuine users pass easily, while automated attack scripts fail.

Q2: "How did you guarantee zero data leakage in feature extraction?"

Answer: All counters, sliding velocity queues, and union-find graph components were computed strictly **causally**: events were sorted chronologically, queried against historical state seen *prior to* event timestamp t_i , and memory state was updated *only after* calculating event i 's features.

Q3: "How would you scale this architecture to 50,000 requests per second in production?"

Answer: (1) An in-memory **Redis Cluster** with sliding-window sorted sets (ZADD/ZREMRANGEBYSCORE) for sub-millisecond velocity lookups. (2) A distributed Union-Find Graph service. (3) Containerized **Triton Inference Server** or FastAPI pods on Kubernetes with auto-scaling to keep inference latency under 10ms. (4) Evidently AI tracking Population Stability Index (PSI) to trigger automated retraining on concept drift.